

Technical Architecture

Our Payment Fraud Detection system is a modular, end-to-end pipeline implemented in Python. We utilize powerful libraries—pandas, scikit-learn, CatBoost, and TabNet via PyTorch—for data processing and modeling, with support for meta-modeling and ensemble learning. Outputs are organized for experiment traceability and future deployment.

Project Flow

- Merge and preprocess raw transaction/identity data
 - Analyze, clean, and balance the dataset
 - Train and validate CatBoost and TabNet models using stratified holds-outs
 - Generate meta-features and train an ensemble stacker for boosted performance
 - Save all core artifacts (models, metrics, importances) for transparency and reproducibility
-

Project Structure

```
.
├── data/
│   ├── raw/                # raw data (if you keep copies)
│   └── processed/
│       └── CleanedData150_Balanced.csv
├── notebooks/
│   ├── DataPreprocessing.ipynb    # feature cleaning/engineering (source)
│   └── HandlingImbalance.ipynb    # class imbalance handling (source)
├── models/                    # saved models (.cbm, .zip, .pkl)
├── results/                   # predictions, metrics, feature importances
├── catboost_train.py           # CatBoost training script
├── tabnet_train.py             # TabNet training script
└── stacking.py                 # Meta-learner training on base predictions
```

Data Collection & Preprocessing

Data Loading and Merging

We began by merging transaction data with corresponding identity datasets to maximize feature coverage.

Code Snippet:

```
python
import pandas as pd
train_transaction = pd.read_csv("train_transaction.csv")
train_identity = pd.read_csv("train_identity.csv")
train_df = train_transaction.merge(train_identity, on="TransactionID", how="left")
```

Data Cleaning

Key steps involved:

- Dropping irrelevant or ID-only columns (e.g., TransactionID)
- Analyzing missing values: No missing labels; missing numerical/categorical values were filled or one-hot encoded as necessary
- Type conversion and label encoding for categorical variables

Code Snippet:

```
python
df_encoded = pd.get_dummies(df_clean, columns=['type'], prefix='type')
X = df_encoded.drop('isFraud', axis=1)
y = df_encoded['isFraud']
```

Balancing the Dataset

Fraud data is typically imbalanced. We address this with undersampling and/or oversampling as explored in `HandlingImbalance.ipynb`. This ensures model learning is not dominated by the majority class.

Exploratory Data Analysis

Univariate and Bivariate Visualizations

Performed using seaborn/matplotlib to plot key distributions and relationships for transaction amount, account balances, and timing.

Code Snippet:

```
python
import matplotlib.pyplot as plt
import seaborn as sns

sns.histplot(data['amount'], bins=50, log_scale=True, kde=True)
```

Model Building

CatBoost Training Pipeline

1. Splitting the Data:

Split data into stratified train/validation sets for fair performance examination.

```
python
from sklearn.model_selection import train_test_split
```

```
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

2. CatBoost Training:

```
python
from catboost import CatBoostClassifier

clf = CatBoostClassifier(
    iterations=4000,
    learning_rate=0.1,
    depth=6,
    loss_function='Logloss',
    eval_metric='AUC',
    early_stopping_rounds=50,
    random_seed=42,
    verbose=100
)
clf.fit(X_train, y_train, eval_set=(X_val, y_val), verbose=50, plot=False)
```

3. Evaluation and Saving:

```
python
from sklearn.metrics import roc_auc_score, classification_report

val_preds = clf.predict_proba(X_val)[:, 1]
roc_auc = roc_auc_score(y_val, val_preds)
print(classification_report(y_val, (val_preds > 0.5).astype(int)))

clf.save_model("models/catboost_model.cbm")
pd.DataFrame({'y_true': y_val, 'y_pred_prob': val_preds}).to_csv(
    "results/catboost_val_predictions.csv", index=False)
```

TabNet Training

TabNet's pipeline mirrors CatBoost, but uses deep learning for tabular feature selection:

```
python
from pytorch_tabnet.tab_model import TabNetClassifier
import torch

clf = TabNetClassifier(
    optimizer_fn = torch.optim.Adam,
    optimizer_params = dict(lr=2e-2),
    mask_type = 'sparsemax',
    seed = 42
)
clf.fit(
    X_train, y_train,
    eval_set=[(X_val, y_val)],
    eval_metric=['auc'],
    max_epochs=25,
    patience=10,
    batch_size=512,
    virtual_batch_size=128,
    num_workers=0,
    drop_last=False
)
```

Model Output/Artifacts

Both scripts automatically save:

- Trained models (.cbm, .zip)

- Validation metrics/reports
 - Feature importances (CSV)
 - Reproducible predictions (CSV)
-

Stacked Model

We combine CatBoost and TabNet using out-of-fold predictions as meta-features in a logistic regression meta-model:

```
python
import pandas as pd
from sklearn.linear_model import LogisticRegression

tabnet_preds = pd.read_csv("results/tabnet_val_predictions.csv")
catboost_preds = pd.read_csv("results/catboost_val_predictions.csv")

X_meta = pd.DataFrame({
    'tabnet': tabnet_preds['y_pred_prob'],
    'catboost': catboost_preds['y_pred_prob']
})
y_meta = tabnet_preds['y_true']

meta_clf = LogisticRegression(max_iter=1000)
meta_clf.fit(X_meta, y_meta)
meta_clf.predict_proba(X_meta)[:, 1]
```

Benefit: Boosts fraud recall by leveraging complementary strengths of CatBoost and TabNet outputs.

Model Evaluation and Feature Insights

CatBoost

- ROC-AUC: 0.9601
- Class 0 (Not Fraud): Precision 0.99, Recall 1.00, F1-score 0.99
- Class 1 (Fraud): Precision 0.95, Recall 0.60, F1-score 0.74
- Conclusion: Strong non-fraud detection; reasonably high fraud precision, with recall limited by class imbalance.

TabNet

- ROC-AUC: 0.8931
- Class 0: Precision 0.97, Recall 1.00, F1-score 0.99
- Class 1: Precision 0.90, Recall 0.27, F1-score 0.42
- Conclusion: Lower overall recall for fraud, but strong calibration for non-fraud, validating the need for ensemble stacking.

Stacked Ensemble (Meta Logistic Regression):

- Class 0: Precision 0.99, Recall 1.00, F1-score 0.99
- Class 1: Precision 0.92, Recall 0.63, F1-score 0.75

- Overall Accuracy: 0.985
- Macro F1: 0.869
- Conclusion: Stacker achieves a strong trade-off, improving recall for fraud cases without sacrificing non-fraud accuracy.

Saving Artifacts and Preparing for Deployment

Every model, scaler, and CSV artifact is saved into dedicated folders. This supports batch scoring, model update audits, and seamless migration into Flask APIs or batch pipelines.

Deployment Preparation & Integration

Saving & Loading Models:

Models are serialized to the models/ directory (catboost_model.cbm, tabnet_model.zip, stacking_meta_model.pkl).

Predictions and feature importance are saved with full provenance for each experiment.

Web/Batch Integration:

The prediction pipeline is designed for easy integration:

Preprocess incoming transactions with the same transformations.

Load model and scaler objects.

Input features are predicted and results (including probability of fraud) are returned for instant decisioning.

Summary & Reflections

- **Methodological rigor:** Stratified validation, reproducible pipelines, and careful artifact management underpin robust, trustworthy results.
- **Feature importance and model stacking:** Offer interpretability for real-world deployments.
- **Ready for production:** Folderized artifacts, validation metrics, and easy model reloads simplify rollout and monitoring.